

CyA14Junio2023.pdf



Anónimo



Computabilidad y Algoritmia



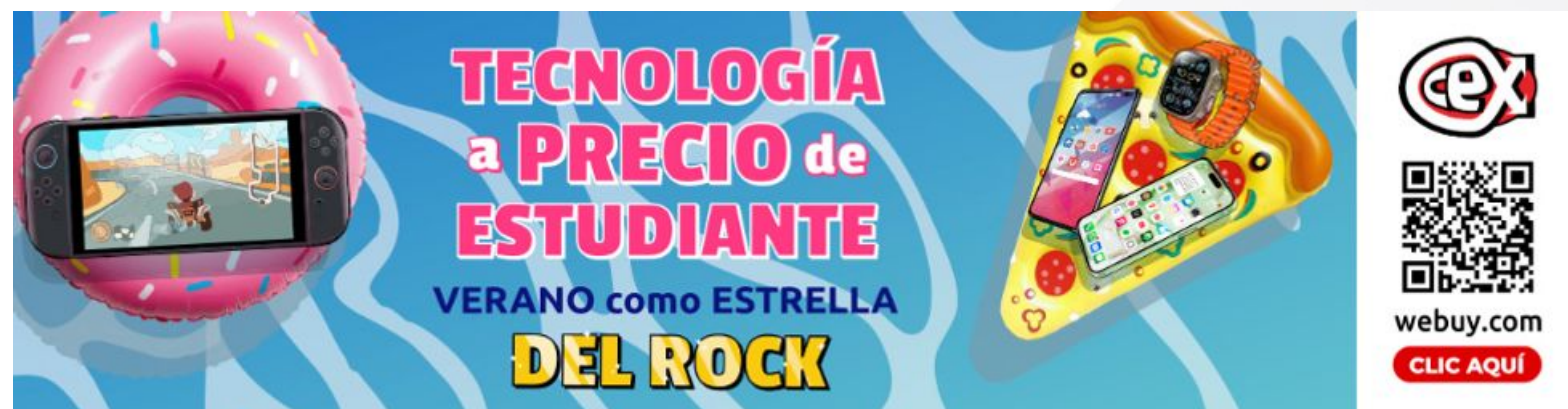
2º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería y Tecnología
Universidad de La Laguna**



[Accede al documento original](#)

A promotional banner for Cex. On the left, a Nintendo Switch is inside a pink inflatable ring with confetti. In the center, text reads 'TECNOLOGÍA a PRECIO de ESTUDIANTE' in large pink letters, followed by 'VERANO como ESTRELLA' in blue and 'DEL ROCK' in large yellow letters. On the right, a yellow inflatable ring contains a smartphone, a smartwatch, and a fitness tracker. To the far right is the Cex logo, a QR code, the website 'webuy.com', and a red button that says 'CLIC AQUÍ'.

20€
Por traerle

Si tienes una Cuenta NoCuenta
te puedes llevar hasta

200€

1/6

Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANK NV se encuentra adherido al Sistema de Garantía de Depósitos Holandes con una garantía de hasta 100.000 euros por depositante. Consulta más información en [ing.es](https://www.ing.es)

(Por cada amigo que traigas hasta 10) Y si no tienes una, háztela* y te puedes llevar 60€

*Promoción exclusiva para nuevos clientes menores de 28 años. Basta con hacer una compra con tarjeta de débito o realizar un Bizum al mes, durante 3 meses. Consulta bases legales en [ing.es](https://www.ing.es)

¡Lo quiero!



do your thing

Escuela Superior
de Ingeniería y Tecnología
Universidad de La Laguna

Computabilidad y Algoritmos
22-2023
5

14 de junio de 2023

DNI: [redacted]
Correo electrónico: [redacted]

LEA DETENIDAMENTE

- No escribir nada en la parte superior.
- El examen consta de 10 preguntas y el tiempo disponible es de 45 minutos.
- Realizar cada ejercicio en un FOLIO DISTINTO y con bolígrafo, y entregar los ejercicios en orden.
- Poner nombre, apellidos y DNI en todos los folios que utilice.
- Al finalizar el examen, ENTREGAR TODOS LOS FOLIOS utilizados, incluyendo este.
- Al entregar el examen muestre algún documento que acredite su identidad.

1. (2 Puntos)

Sea L el lenguaje formado por todas las cadenas binarias w , con $|w| \equiv 4 \pmod{4}$, en las que cada bloque de cuatro símbolos contiene al menos dos ceros consecutivos. Los bloques comenzarán siempre en posiciones múltiplo de 4.

- Obtener una expresión regular que represente a L . (0.5 puntos)
- Obtener un autómata finito que reconozca L . (1 punto)
- ¿Existe una gramática independiente del contexto que genere el lenguaje L ? En caso afirmativo, indique cuál sería. En caso contrario, justifique su respuesta. (0.5 puntos)

2. (2 Puntos)

Sea L un lenguaje independiente del contexto pero no regular, y sea L' otro lenguaje tal que:

- $L' \cap L$ es infinito
- $L' \neq L$

Demuestre la verdad o falsedad de las siguientes afirmaciones:

- L' no es regular.
- Dado cualquier lenguaje L independiente del contexto pero no regular, siempre existe un lenguaje L' regular que cumple (i) y (ii).

3. (2 Puntos)

Construir una máquina de Turing que dada una cadena binaria de entrada, añada a la cadena un 0 entre cada pareja de 1's.

Antes de diseñar la máquina, explicar brevemente el modo de funcionamiento que se planea para la misma. Para cada transición o conjunto de transiciones de la máquina de Turing describir lo que se pretende hacer con las transiciones en cuestión. Dibujar el diagrama de transiciones de la máquina.

Camino San Francisco de Paula, s/n. Apartado 456. 38200 San Cristóbal de La Laguna. S/C de Tenerife

4. (2 Puntos)

Dada la clase `matrix_t<T>`, con los siguientes métodos ya desarrollados:

- `matrix_t(const int m=0, const int n=0)`: constructor, m filas y n columnas.
- `int get_m(void)` y `int get_n(void)`: *getters* del número de filas y columnas, respectivamente.
- `T& operator()(const int i, const int j)`: operador de indexación de la matriz en la fila i y la columna j .
- `matrix_t<T> submatrix(const int r_ini, const int r_end, const int c_ini, const int c_end)`: extrae una sub-matriz de la matriz invocante entre las filas r_ini y r_end , y las columnas c_ini y c_end .
- `matrix_t<T>& operator=(const matrix_t<T>& A)`: operador de asignación.
- `matrix_t<T> operator+(const matrix_t<T>& A, const matrix_t<T>& B)`: operador externo que devuelve la suma de dos matrices A y B .
- `matrix_t<T> operator-(const matrix_t<T>& A, const matrix_t<T>& B)`: operador externo que devuelve la resta (diferencia) de dos matrices A y B .

Por otro lado, dadas dos matrices A y B cuadradas de tamaño $N \times N$, donde $N = A.get_m()$, con un número de filas/columnas múltiplo de 2^N , con $N > 0$, el algoritmo de Strassen para la multiplicación de matrices tipo **Divide-y-Vencerás** consiste en dividir A y B en cuatro (4) sub-matrices de tamaño $N/2 \times N/2$ como se muestra en la siguiente figura:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}_A \times \begin{bmatrix} e & f \\ g & h \end{bmatrix}_B = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}_C$$

Donde:

- A , B y C son matrices cuadradas de tamaño $N \times N$
- a , b , c y d son sub-matrices de A , de tamaño $N/2 \times N/2$
- e , f , g y h son sub-matrices de B , de tamaño $N/2 \times N/2$
- $p1 = a * (f - h)$
- $p2 = (a + b) * h$
- $p3 = (c + d) * e$
- $p4 = d * (g - e)$
- $p5 = (a + d) * (e + h)$
- $p6 = (b - d) * (g + h)$
- $p7 = (a - c) * (e + f)$
- Las filas y columnas se indexan desde 1

Se pide:

- (1) Desarrollar el algoritmo de Strassen **recursivo** (en pseudo-C++) de tipo **Divide-y-Vencerás** que implemente la multiplicación de matrices, y que tenga la siguiente cabecera: (1.5 puntos)

`matrix_t<T>
Strassen(const matrix_t<T>& A, const matrix_t<T>& B)`

- (2) Calcular la tasa de crecimiento $T(N)$ del algoritmo del apartado anterior y, usando el Teorema Maestro, la complejidad en notación $\Theta(\cdot)$. ¿Es mejor este algoritmo recursivo que el clásico método de $\Theta(N^3)$? (0.5 puntos)

Nota: En la corrección de este ejercicio se tendrá muy en cuenta la eficiencia en la implementación y su diseño.

5. 2 puntos

Contestar Verdadero (V) o Falso (F) o elegir la/s respuesta/s correcta/s para cada una de las siguientes preguntas.

NOTAS ACLARATORIAS:

- Por cada respuesta correcta se sumará 0.2 puntos.
- Por cada respuesta incorrecta se restará 0.2 puntos.
- Las preguntas sin responder se considerarán incorrectas.
- La puntuación mínima de esta pregunta es de 0 puntos.

- ✓ (1) $\emptyset^0 = \emptyset$ **F**
- ✓ (2) Si L no cumple el lema del bombeo para lenguajes regulares $\Rightarrow L$ no es regular **V**
- ✓ (3) Los lenguajes regulares son cerrados respecto a la intersección **V**
- ✓ (4) Si L_1 y L_2 son lenguajes independientes del contexto $\Rightarrow L_1 L_2$ también lo es **V**
- ✓ (5) El resultado de concatenar un lenguaje regular con uno independiente del contexto es siempre independiente del contexto **V**
- ✗ (6) Si L es recursivamente enumerable $\Rightarrow \Sigma^* - L$ es recursivamente enumerable **F**
- ✓ (7) Si L es un lenguaje independiente del contexto entonces es recursivamente enumerable **V**
- ✗ (8) $5n + 8n^2 + 100n^3 = O(n^4)$ **F**
- ✗ (9) $T(n) = 16T(n/4) + n = \Theta(n)$ **V**
- ✗ (10) La complejidad del siguiente bloque de código es $\Theta(n^3)$ **F**
- ```
int suma = 0;
for (int a = 1; a <= n; a++)
 for (int b = 1; b <= n; b++)
 for (int c = 1; c <= b; c++)
 suma++;
```

# COMPRA SMART en



CLIC AQUÍ

EL VERANO

webuy.com

DISFRUTA HARD

```
template<class T>
matrix_t<T>
operator*(const matrix_t<T>& A, const matrix_t<T>& B)
{
 const int N = A.get_m();

 matrix_t<T> C(N, N);

 //if (N == 1) C(1, 1) = A(1, 1) * B(1, 1);

 if (N == 2)
 {
 C(1, 1) = A(1, 1) * B(1, 1) + A(1, 2) * B(2, 1);
 C(1, 2) = A(1, 1) * B(1, 2) + A(1, 2) * B(2, 2);
 C(2, 1) = A(2, 1) * B(1, 1) + A(2, 2) * B(2, 1);
 C(2, 2) = A(2, 1) * B(1, 2) + A(2, 2) * B(2, 2);
 }
 else
 {
 matrix_t<T> a = A.submatrix(1, N/2, 1, N/2),
 b = A.submatrix(1, N/2, N/2 + 1, N),
 c = A.submatrix(N/2 + 1, N, 1, N/2),
 d = A.submatrix(N/2 + 1, N, N/2 + 1, N),
 e = B.submatrix(1, N/2, 1, N/2),
 f = B.submatrix(1, N/2, N/2 + 1, N),
 g = B.submatrix(N/2 + 1, N, 1, N/2),
 h = B.submatrix(N/2 + 1, N, N/2 + 1, N);

 matrix_t<T> ae_bg = a * e + b * g,
 af_bh = a * f + b * h,
 ce_dg = c * e + d * g,
 cf_dh = c * f + d * h;

 /*
 for (int i = 1; i <= N; ++i)
 for (int j = 1; j <= N; ++j)
 if (i <= N/2 && j <= N/2) C(i, j) = ae_bg(i, j);
 else if (i <= N/2 && j > N/2) C(i, j) = af_bh(i, j - N/2);
 else if (i > N/2 && j <= N/2) C(i, j) = ce_dg(i - N/2, j);
 else C(i, j) = cf_dh(i - N/2, j - N/2);
 */
 for (int i = 1; i <= N/2; ++i)
 for (int j = 1; j <= N/2; ++j)
 {
 C(i, j) = ae_bg(i, j);
 C(i, j + N/2) = af_bh(i, j);
 C(i + N/2, j) = ce_dg(i, j);
 C(i + N/2, j + N/2) = cf_dh(i, j);
 }
 }
 return C;
}
```

b. Sabiendo que la suma de dos matrices tiene una complejidad de  $\Theta(N^2)$ , calcular la tasa de crecimiento  $T(N)$  del algoritmo del apartado anterior y, usando el Teorema Maestro, la complejidad en notación  $\Theta(\cdot)$ . ¿Es mejor este algoritmo recursivo que el clásico método de  $\Theta(N^3)$ ?

**Solución:**

$$T(N) = 8T(N/2) + N^2 \Rightarrow p = 8 > d^k = 2^2 = 4 \Rightarrow \Theta(N^{\log_2 8}) = \Theta(N^3)$$

9. [Pregunta de examen] Dada la clase **matrix\_t<T>**, con los siguientes métodos ya desarrollados:

- **matrix\_t(const int m=0, const int n=0):** constructor, *m* filas y *n* columnas.
- **int get\_m(void)** y **int get\_n(void):** getters del número de filas y columnas, respectivamente.
- **T& operator()(const int i, const int j):** operador de indexación de la matriz en la fila *i* y la columna *j*.

WUOLAH

- `matrix_t<T> submatrix(const int r_ini, const int r_end, const int c_ini, const int c_end)`: extrae una sub-matriz de la matriz invocante entre las filas **r\_ini** y **r\_end**, y las columnas **c\_ini** y **c\_end**.
- `matrix_t<T>& operator=(const matrix_t<T>& A)`: operador de asignación.
- `matrix_t<T> operator+(const matrix_t<T>& A, const matrix_t<T>& B)`: operador externo que devuelve la suma de dos matrices *A* y *B*.
- `matrix_t<T> operator-(const matrix_t<T>& A, const matrix_t<T>& B)`: operador externo que devuelve la resta (diferencia) de dos matrices *A* y *B*.

Por otro lado, dadas dos matrices *A* y *B* cuadradas de tamaño  $N \times N$ , donde  $N = A.get\_m()$ , con un número de filas/columnas múltiplo de  $2^N$ , con  $N > 0$ , el algoritmo de **multiplicación de matrices tipo Divide-y-Vencerás** consiste en dividir *A* y *B* en cuatro (4) sub-matrices de tamaño  $N/2 \times N/2$  como se muestra en la siguiente figura:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

A                      B                      C

Donde:

- *A*, *B* y *C* son matrices cuadradas de tamaño  $N \times N$
- *a*, *b*, *c* y *d* son sub-matrices de *A*, de tamaño  $N/2 \times N/2$
- *e*, *f*, *g* y *h* son sub-matrices de *B*, de tamaño  $N/2 \times N/2$
- $p1 = a * (f - h)$
- $p2 = (a + b) * h$
- $p3 = (c + d) * e$
- $p4 = d * (g - e)$
- $p5 = (a + d) * (e + h)$
- $p6 = (b - d) * (g + h)$
- $p7 = (a - c) * (e + f)$
- Las filas y columnas se indexan desde 1

Se pide:

- Desarrollar el algoritmo de Strassen **recursivo** (en pseudo-C++) tipo **Divide-y-Vencerás** que implemente la multiplicación de matrices, y que tenga la siguiente cabecera:

```
matrix_t<T> Strassen(const matrix_t<T>& A, const matrix_t<T>& B)
```

**Solución:**



piensa  
en tu futuro  
desde ya.

únete a la comunidad randstad ingeniería  
con las mejores ofertas de empleo



unirme a  
mi comunidad

```

template<class T>
matrix_t<T>
Strassen(const matrix_t<T>& A, const matrix_t<T>& B)
{
 const int N = A.get_m();

 matrix_t<T> C(N, N);

/*
 if (N == 2)
 {
 C(1, 1) = A(1, 1) * B(1, 1) + A(1, 2) * B(2, 1);
 C(1, 2) = A(1, 1) * B(1, 2) + A(1, 2) * B(2, 2);
 C(2, 1) = A(2, 1) * B(1, 1) + A(2, 2) * B(2, 1);
 C(2, 2) = A(2, 1) * B(1, 2) + A(2, 2) * B(2, 2);
 }
*/

 if (N == 1) C(1, 1) = A(1, 1) * B(1, 1);
 else
 {
 matrix_t<T> a = A.submatrix(1, N/2, 1, N/2),
 b = A.submatrix(1, N/2, N/2 + 1, N),
 c = A.submatrix(N/2 + 1, N, 1, N/2),
 d = A.submatrix(N/2 + 1, N, N/2 + 1, N),
 e = B.submatrix(1, N/2, 1, N/2),
 f = B.submatrix(1, N/2, N/2 + 1, N),
 g = B.submatrix(N/2 + 1, N, 1, N/2),
 h = B.submatrix(N/2 + 1, N, N/2 + 1, N);

 matrix_t<T> p1 = Strassen(a, f - h),
 p2 = Strassen(a + b, h),
 p3 = Strassen(c + d, e),
 p4 = Strassen(d, g - e),
 p5 = Strassen(a + d, e + h),
 p6 = Strassen(b - d, g + h),
 p7 = Strassen(a - c, e + f),
 p5_p4_p2_p6 = p5 + p4 - p2 + p6,
 p1_p2 = p1 + p2,
 p3_p4 = p3 + p4,
 p1_p5_p3_p7 = p1 + p5 - p3 - p7;

/*
 for (int i = 1; i <= N; ++i)
 for (int j = 1; j <= N; ++j)
 if (i <= N/2 && j <= N/2) C(i, j) = p5_p4_p2_p6(i, j);
 else if (i <= N/2 && j > N/2) C(i, j) = p1_p2(i, j - N/2);
 else if (i > N/2 && j <= N/2) C(i, j) = p3_p4(i - N/2, j);
 else C(i, j) = p1_p5_p3_p7(i - N/2, j - N/2);
*/

 for (int i = 1; i <= N/2; ++i)
 for (int j = 1; j <= N/2; ++j)
 {
 C(i, j) = p5_p4_p2_p6(i, j);
 C(i, j + N/2) = p1_p2(i, j);
 C(i + N/2, j) = p3_p4(i, j);
 C(i + N/2, j + N/2) = p1_p5_p3_p7(i, j);
 }
 }

 return C;
}

```

b. Calcular la tasa de crecimiento  $T(N)$  del algoritmo del apartado anterior y, usando el Teorema Maestro, la complejidad en notación  $\Theta(\cdot)$ .  
¿Es mejor este algoritmo recursivo que el clásico método de  $\Theta(N^3)$ ?

**Solución:**

$$T(N) = 7T(N/2) + N^2 \Rightarrow p = 7 > d^k = 2^2 = 4 \Rightarrow \Theta(N^{\log_2 7}) = \Theta(N^{2.8})$$